Lab 22

Building a GitHub Actions CI/CD Pipeline with API Controller

WSO2 API Manager 4.7.0 · Practical exercise

Before You Begin

This lab continues the CI/CD with Git lab (Lab 21). URLs and paths use placeholders. Replace them with your own values:

- <WORK_DIR> – the working folder from Lab 21, which contains the Source and Deployment repositories.
- <APICTL_DIR> – the full path of the folder that contains the apictl program, for example /Users/<you>/apictl (Windows: C:/apictl).
- <APIM_HOST>, <HTTPS_PORT>, <PROD_HTTPS_PORT> – as in Lab 21 (dev 9443, prod 9445 with offset 2).
- <OWNER> – your GitHub user name. <REPO> – the repository you create in Part 1, for example api-cicd.

The API names in this guide are those of Lab 21 (SwaggerPetstore, version 1.0.0). If your API has another name or version, for example SwaggerPetstore-OpenAPI30 1.0.27, use your names.

Training Objective

Learn how to run the Lab 21 deployment automatically with GitHub Actions: every push to GitHub deploys the changed APIs to dev, and a button promotes them to prod, using apictl vcs and environment values kept in Git.

Business Scenario

PizzaShack keeps its code on GitHub and wants API changes to reach dev without manual work after every commit. Production changes must be started on purpose by a person, and prod back-end URLs must be managed separately from the API definition.

How It Works

GitHub Actions runs a workflow (a YAML file in .github/workflows) when something happens in the repository, for example a push. The work is done by a runner. GitHub's own runners are in the cloud and cannot reach an API Manager on your laptop or inside a company network, so in this lab you run a self-hosted runner on your laptop. The runner connects out to GitHub, picks up jobs, and runs apictl locally.

```
You --git push--> GitHub repository <REPO>
                    main branch:        API projects + workflows
                    deployment branch:  DeploymentArtifacts_* (params.yaml)
                          |
                    workflow "Deploy APIs to dev"   (on every push to main)
                    workflow "Deploy APIs to prod"  (Run workflow button)
                          |
                 self-hosted runner on your laptop
                    apictl vcs deploy -e dev / -e prod
                          |
                 API Manager dev (<HTTPS_PORT>) and prod (<PROD_HTTPS_PORT>)
```

CC BY 4.0

Lab 21 used two Git repositories. In GitHub, this lab keeps both in one repository, as two branches. Keep them separate: apictl vcs needs its vcs.yaml at the root of each repository, and it treats any project folder in the repository as an API to deploy. The workflow checks out the two branches into two folders, `source` and `deployment`, exactly like the two Lab 21 folders.

High-Level Steps

- Create a private GitHub repository and push the Lab 21 Source and Deployment repositories to it as two branches
- Store the API Manager user and password as GitHub secrets, and the URLs as variables
- Install and start a self-hosted runner on your laptop
- Add the two workflows and watch the first deployment to dev
- Change the API on GitHub, see it deployed to dev, and promote it to prod

Prerequisites

- Lab 21 completed: SwaggerPetstore exists in dev and prod, and <WORK_DIR>/Source and <WORK_DIR>/Deployment are Git repositories that contain vcs.yaml.
- A GitHub account, and internet access to github.com from your laptop (check with your network team if a proxy blocks it).
- Git and apictl 4.7.x on your laptop. Both API Manager instances running.
- The two workflow files from the lab files: `03 Lab Files/github-actions/deploy-dev.yml` and `deploy-prod.yml`. They are also printed in the appendix.

Part 1 – Create the GitHub Repository

1. Sign in to GitHub, click **New repository**, enter `<REPO>` as the name, select **Private**, leave every "Initialize" option empty, and click **Create repository**.

Keep the repository private. A self-hosted runner runs the workflow code on your laptop. In a public repository, anyone could open a pull request that runs code on your machine.

2. Push the Lab 21 Source repository as the **main** branch:

macOS / Linux / Git Bash

```
cd <WORK_DIR>/Source
git branch -M main
git remote add origin https://github.com/<OWNER>/<REPO>.git
git push -u origin main
```

If Git asks for a password, GitHub needs a personal access token, not your account password. The simplest way is the GitHub CLI: run `gh auth login` and then `gh auth setup-git`, and push again. On Windows, Git Credential Manager opens a browser sign-in.

3. Push the Lab 21 Deployment repository to the same GitHub repository, as a branch named **deployment**:

```
cd <WORK_DIR>/Deployment
git remote add origin https://github.com/<OWNER>/<REPO>.git
git push origin HEAD:deployment
```

CC BY 4.0

4. On GitHub, refresh the repository page and open the branch list. You see two branches:

Branch	Content
main	vcs.yaml and the folder SwaggerPetstore-1.0.0
deployment	vcs.yaml and the folder DeploymentArtifacts_SwaggerPetstore-1.0.0 with params.yaml (dev and prod URLs)

Part 2 – Add Secrets and Variables

5. In the repository, go to **Settings** → **Secrets and variables** → **Actions**. On the **Secrets** tab, click **New repository secret** and add:

Name	Value
`APIM_USERNAME`	admin
`APIM_PASSWORD`	The API Manager admin password

   GitHub stores secrets encrypted and hides them in the workflow logs. Never write a password in a workflow file.

6. On the **Variables** tab, click **New repository variable** and add these three. Use full paths; the runner does not expand ~.

Name	Value
`APICTL_DIR`	`<APICTL_DIR>`
`APIM_DEV_URL`	`https://<APIM_HOST>:<HTTPS_PORT>`
`APIM_PROD_URL`	`https://<APIM_HOST>:<PROD_HTTPS_PORT>`

Part 3 – Install a Self-Hosted Runner

7. Go to **Settings** → **Actions** → **Runners** and click **New self-hosted runner**. Select your operating system and architecture (for example macOS / ARM64 for an Apple silicon Mac, or Windows / x64).

8. GitHub shows a **Download** and a **Configure** block with commands for your machine. Run them in a new terminal, one by one. They create an `actions-runner` folder, download the runner, and register it with a one-time token. The configure command looks like this:

   macOS / Linux

```
./config.sh --url https://github.com/<OWNER>/<REPO> --token <TOKEN_SHOWN_BY_GITHUB>
```

   Windows (PowerShell)

```
.\config.cmd --url https://github.com/<OWNER>/<REPO> --token <TOKEN_SHOWN_BY_GITHUB>
```

   Press Enter to accept every default (runner group, name, labels and work folder). The token is valid for one hour; if it has expired, open the New self-hosted runner page again to get a new one.

CC BY 4.0

9. Start the runner and keep the terminal open:

macOS / Linux

```
./run.sh
```

Windows (PowerShell)

```
.\run.cmd
```

Expected: `Connected to GitHub` and `Listening for Jobs`. In **Settings** → **Actions** → **Runners** the runner shows as **Idle**.

Windows: the workflow steps use bash. Git Bash must be on the PATH before the Windows bash of WSL (C:\Windows\System32\bash.exe); add C:\Program Files\Git\bin to the PATH and start the runner from a new window.

Part 4 – Add the Workflows

10. In your Source repository, create the folder `.github/workflows` and copy the two workflow files into it:

macOS / Linux / Git Bash

```
cd <WORK_DIR>/Source
mkdir -p .github/workflows
cp <LAB_FILES>/github-actions/deploy-dev.yml  .github/workflows/
cp <LAB_FILES>/github-actions/deploy-prod.yml .github/workflows/
```

`<LAB_FILES>` is the folder 03 Lab Files of this training. The two workflows are the same except for the trigger and the environment:

	deploy-dev.yml	**deploy-prod.yml**
Starts	On every push to main, and with Run workflow	Only with Run workflow
apictl environment	dev	prod
API Manager URL	vars.APIM_DEV_URL	vars.APIM_PROD_URL

Each workflow has these steps:

Step	**What it does**
Check out API projects (main)	Clones the main branch into the folder source. fetch-depth: 0 gets the full history, because apictl compares the files with the commit it deployed last time.
Check out environment values	Clones the deployment branch into the folder deployment.
Add apictl to the PATH	Adds vars.APICTL_DIR to the PATH of the next steps.
Configure apictl and log in	Adds the environment to apictl if it is missing, points apictl vcs at the two folders, and logs in with the secrets.
Show the API projects to deploy	apictl vcs status, the same command as in Lab 21
Deploy	apictl vcs deploy, the same command as in Lab 21

CC BY 4.0

11. Commit and push the workflows:

```
git add .github
git commit -m "Add deployment workflows"
git push
```

Part 5 – Watch the First Deployment

12. The push starts **Deploy APIs to dev**. On GitHub, open the **Actions** tab and click the running workflow, then the **deploy** job. The runner terminal shows `Running job: deploy`.

13. Open the **Show the API projects to deploy** and **Deploy** steps. Expected (the first time apictl deploys this repository to dev, every project is deployed):

```
Projects to Deploy (1)
APIs (1) ...
1: [save]    SwaggerPetstore-1.0.0: (SwaggerPetstore-1.0.0)
...
Successfully imported API.
```

If the step shows `Everything is up-to-date`, apictl on this laptop has already deployed this commit to dev. That is also a success.

14. In the dev Publisher, open SwaggerPetstore and go to **Deployments**. A new revision has been deployed, and the endpoints are the dev values from params.yaml.

Part 6 – Change the API on GitHub

15. On GitHub, open the main branch, then `SwaggerPetstore-1.0.0/api.yaml`, and click the pencil icon. Find the line `description:` under `data:` and change it, for example:

```
  description: "Petstore API, deployed by GitHub Actions"
```

If the line is `description: null` or missing, add it under `name:` with two spaces of indentation, as shown.

16. Click **Commit changes**, keep **Commit directly to the main branch**, and click **Commit changes** again.

17. On the **Actions** tab, a new run of Deploy APIs to dev starts by itself. The status step lists SwaggerPetstore-1.0.0, and the deploy step prints `Successfully imported API.` The dev Publisher shows the new description; prod still shows the old one.

Pull the change to your laptop before you edit files locally again: `git -C <WORK_DIR>/Source pull`.

CC BY 4.0

Part 7 – Promote to Prod

18. On the **Actions** tab, click **Deploy APIs to prod** in the left list, then **Run workflow**, keep the branch **main**, and click the green **Run workflow** button.

19. When the run finishes, check the prod Publisher. SwaggerPetstore has the new description, and its endpoints are still the prod values from params.yaml:

Environment	Expected production endpoint
dev	`https://dev.wso2.com`
prod	`https://prod.wso2.com`

20. Now change a prod value only. On GitHub, switch to the **deployment** branch, edit `DeploymentArtifacts_SwaggerPetstore-1.0.0/params.yaml`, change the prod production URL to `https://prod2.wso2.com` and commit it to the deployment branch.

21. A push to the deployment branch does not start a workflow, because the workflows are on main. Run **Deploy APIs to prod** again with **Run workflow**. The status step prints `Everything is up-to-date`, because no API project changed, but the deploy step still deploys `DeploymentArtifacts_SwaggerPetstore-1.0.0`, because its parameters changed. The prod endpoint is now prod2.wso2.com.

Part 8 – Optional: Require Approval for Prod

22. GitHub can make a person approve a job before it runs. Go to **Settings** → **Environments**, click **New environment**, name it `production`, select **Required reviewers**, add yourself and click **Save protection rules**.

Required reviewers is available for public repositories, and for private repositories on paid GitHub plans (for example Team or Enterprise). If the option is not shown, your plan does not include it; the Run workflow button of Part 7 is then the manual gate.

23. In `deploy-prod.yml`, add the line below under `runs-on: self-hosted`, commit and push. The next prod run waits for **Review deployments** → **Approve and deploy**.

```
    environment: production
```

24. Optional: change the trigger of deploy-prod.yml to a push of a tag, so that prod is deployed for released versions only:

```
on:
  push:
    tags: ["v*"]
```

Then `git tag v1.0 && git push origin v1.0` starts the prod deployment.

Moving to a Production Setup

- Run the self-hosted runner as a service on a server inside the company network that can reach the API Manager servers (./svc.sh install on Linux and macOS, or the service option of config.cmd on Windows), not on a laptop.

CC BY 4.0

- Use a dedicated API Manager user for deployments with only the publisher and import/export permissions, instead of admin, and one set of secrets per environment (GitHub environment secrets).
- Protect the main and deployment branches: changes through pull requests with review, and only the operations team can change the deployment branch.
- Add a governance check before the deploy step with apictl import api --dry-run, as in the Jenkins lab.
- Use trusted certificates on API Manager and remove -k.

Troubleshooting

Symptom	Cause and fix
Job stays at "Waiting for a runner to pick up this job"	The runner is not running. Start ./run.sh (Windows: run.cmd) and check that it shows Idle in Settings → Actions → Runners.
apictl: command not found	The APICTL_DIR variable is missing or is not the full path of the folder that contains apictl.
fatal: bad object <commit id>	The source checkout is missing the history. Keep fetch-depth: 0 on both checkout steps.
vcs.yaml is not found in the repository root	vcs init was not run in the Lab 21 repository that was pushed to this branch. Run apictl vcs init there, commit vcs.yaml and push.
Environment 'prod' does not exist in .../DeploymentArtifacts_...	params.yaml on the deployment branch has no section with name: prod. Check the environment names and indentation.
Login fails (401)	Wrong APIM_USERNAME or APIM_PASSWORD secret, or wrong URL variable.
The run succeeds with Everything is up-to-date, but the API is missing in that environment	apictl remembers the last commit it deployed to each environment, not what is on the server. If the API was deleted by hand, change the API in Git (Part 6), or remove that environment's entry from ~/.wso2apictl/vcs_config.yaml on the runner machine and run the workflow again.
Scope already assigned locally by another API (400)	Another API in that environment uses the same scope names, for example a second copy of Petstore. Delete the other copy or rename its scopes.

Clean-Up

25. Stop the runner with Ctrl+C. To remove it from GitHub, open **Settings → Actions → Runners**, click the runner, click **Remove**, and run the remove command shown there in the actions-runner folder.

26. The workflows changed the apictl vcs folder settings on your laptop. To use the Lab 21 commands again, set them back:

```
apictl set --vcs-source-repo-path <WORK_DIR>/Source
apictl set --vcs-deployment-repo-path <WORK_DIR>/Deployment
```

Expected Outcome

API projects and environment values from Lab 21 live in a GitHub repository. Every push to main deploys the changed APIs to dev through a self-hosted runner, and a Run workflow button promotes them to prod, with prod values from the deployment branch and credentials kept in GitHub secrets.

CC BY 4.0

Appendix A – .github/workflows/deploy-dev.yml

```
# Deploys the API projects that changed since the last dev deployment.
# API projects come from the main branch; environment values (params.yaml) from the
deployment branch.
name: Deploy APIs to dev

on:
  push:
    branches: [main]      # every push to main deploys the changed APIs to dev
  workflow_dispatch:      # "Run workflow" button, for example after a params.yaml change

concurrency: apictl       # apictl keeps its settings on the runner, so run one
deployment at a time

jobs:
  deploy:
    runs-on: self-hosted
    env:
      APIM_ENV: dev
      APIM_URL: ${{ vars.APIM_DEV_URL }}
      APIM_USERNAME: ${{ secrets.APIM_USERNAME }}
      APIM_PASSWORD: ${{ secrets.APIM_PASSWORD }}
    defaults:
      run:
        shell: bash

    steps:
      - name: Check out API projects (main)
        uses: actions/checkout@v4
        with:
          path: source
          fetch-depth: 0    # apictl compares with the last deployed commit, so the full
history is needed
```

```
      - name: Check out environment values (deployment branch)
        uses: actions/checkout@v4
        with:
          ref: deployment
          path: deployment
          fetch-depth: 0

      - name: Add apictl to the PATH
        run: echo "${{ vars.APICTL_DIR }}" >> "$GITHUB_PATH"

      - name: Configure apictl and log in
        run: |
          envs=$(apictl get envs --format "{{.Name}}")
          grep -qx "$APIM_ENV" <<< "$envs" || apictl add env "$APIM_ENV" --apim
"$APIM_URL"
          apictl set --vcs-source-repo-path "$GITHUB_WORKSPACE/source"
          apictl set --vcs-deployment-repo-path "$GITHUB_WORKSPACE/deployment"
          echo "$APIM_PASSWORD" | apictl login "$APIM_ENV" -u "$APIM_USERNAME" --
password-stdin -k

      - name: Show the API projects to deploy
        working-directory: source
        run: apictl vcs status -e "$APIM_ENV"

      - name: Deploy
        working-directory: source
        run: apictl vcs deploy -e "$APIM_ENV" -k
```

CC BY 4.0

Appendix B – .github/workflows/deploy-prod.yml

```
# Promotes the API projects that changed since the last prod deployment.
# API projects come from the main branch; environment values (params.yaml) from the
deployment branch.
name: Deploy APIs to prod

on:
  workflow_dispatch:      # prod is deployed only when someone clicks "Run workflow"

concurrency: apictl       # apictl keeps its settings on the runner, so run one
deployment at a time

jobs:
  deploy:
    runs-on: self-hosted
    env:
      APIM_ENV: prod
      APIM_URL: ${{ vars.APIM_PROD_URL }}
      APIM_USERNAME: ${{ secrets.APIM_USERNAME }}
      APIM_PASSWORD: ${{ secrets.APIM_PASSWORD }}
    defaults:
      run:
        shell: bash

    steps:
      - name: Check out API projects (main)
        uses: actions/checkout@v4
        with:
          path: source
          fetch-depth: 0    # apictl compares with the last deployed commit, so the full
history is needed
```

```
      - name: Check out environment values (deployment branch)
        uses: actions/checkout@v4
        with:
          ref: deployment
          path: deployment
          fetch-depth: 0

      - name: Add apictl to the PATH
        run: echo "${{ vars.APICTL_DIR }}" >> "$GITHUB_PATH"

      - name: Configure apictl and log in
        run: |
          envs=$(apictl get envs --format "{{.Name}}")
          grep -qx "$APIM_ENV" <<< "$envs" || apictl add env "$APIM_ENV" --apim
"$APIM_URL"
          apictl set --vcs-source-repo-path "$GITHUB_WORKSPACE/source"
          apictl set --vcs-deployment-repo-path "$GITHUB_WORKSPACE/deployment"
          echo "$APIM_PASSWORD" | apictl login "$APIM_ENV" -u "$APIM_USERNAME" --
password-stdin -k

      - name: Show the API projects to deploy
        working-directory: source
        run: apictl vcs status -e "$APIM_ENV"

      - name: Deploy
        working-directory: source
        run: apictl vcs deploy -e "$APIM_ENV" -k
```

CC BY 4.0